

Fundamentals

1. N-Tier Architecture:

N-tier architecture is a pattern which divides the application into layers, to separate the responsibility of each layer. Presentation layer, Business layer and Data layer. The presentation layer receives the request from the user, the business layer validates it and the data layer is responsible for storing it in the database. The presentation layer never touches SQL, the business layer never shows UI and the data layer never validates data (user input). Each layer has exactly one job.

People think:

N-tier = 3 folders

No,

N-tier = Separating responsibilities

2. Clean Architecture:

Clean architecture is a pattern that works on Single Responsibility Principle (SRP). It states that, "Each class should have one clear responsibility, and each layer should only do what it is meant to do."

It organizes the project into separate layers with clearly defined responsibilities. Such as Presentation(Controller), Application(Use-Cases), Domain(Entities) and Infrastructure(Data access). In clean architecture, every layer points inwards towards the domain layer.

The real goal of clean architecture is, "business rules should survive changes in UI, database, frameworks and external technologies."

3. SOLID Principles:

SOLID is not an architecture. It is a set of five object-oriented design principles that help us write code that is:

- Easy to maintain
- Easy to extend
- Easy to test
- Less tightly coupled
- More reusable

SOLID stands for:

S-Single Responsibility Principle(SRP): A class should have only one reason to change.

O-Open/Closed Principle(OCP): Software entities should be open for extension but closed for modification.

L-Liskov Substitution Principle(LSP): Every child class should satisfy the contract of parent class.

I-Interface Segregation Principle(ISP): A class should not be forced to depend on or implement methods it does not use.

D-Dependency Inversion Principle(DIP): High-level modules should not depend on low-level modules. Both should depend on abstractions.

4. Dependency Injection:

Dependency Injection(DI) is a technique that provides the dependency of a class from outside. It creates the object of classes and injects them into classes that depend on that object.

It uses .NET built-in DI container to create these objects and inject them where needed, resolve dependencies and manage their lifetimes. DI also makes testing easier, because dependencies are injected from outside, we can swap real implementations with fake ones in tests without changing the class itself.

5. DI Lifetimes:

DI lifetimes control how long an object exists in the memory and when a new instance is created. In .NET there are three primary lifetimes:

1. Transient
2. Scoped
3. Singleton

1. Transient: Creates a brand-new object every single time it is requested.

Analogy: A vending machine. Every time you push the button, you get a fresh, separate item.

Best for: Lightweight, stateless services (e.g., calculators, formatting utilities).

2. Scoped: Creates one object per HTTP request. Within that request, the same instance is shared across all components. It is best for web applications where operations should be shared during a single HTTP request but isolated from other users.

Analogy: A waiter assigned to your table. They serve everyone at your table for the duration of the meal, but other tables have their own waiter.

Best for: Services holding state for a single request (e.g., Database Contexts/DbContext, user authentication context).

3. Singleton: Creates a single object the very first time it is requested. That exact same instance is shared across the entire application for its whole lifespan.

Analogy: The restaurant's front door or host stand. Every single guest uses the exact same one, no matter when they arrive or what table they sit at.

Best for: Global, shared resources that are thread-safe (e.g., application configuration caching, logging engines).

Captive Dependency Bug:

The Rule: Never inject a Scoped service into a Singleton service.

The Problem: The Singleton lives forever, so it traps the Scoped service and never lets it close.

The Danger: If that Scoped service is a database connection, it can leak data between different users or crash your app.

6. Dependency Inversion Principle:

It states that **high-level** modules should not depend on **low-level** modules. Both should depend on **abstractions**(interfaces or abstract classes). While **abstractions** should not depend on **details**(concrete classes), **details** should depend on **abstractions**.

Data Access

7. Repository Pattern:

It is a designed pattern that abstracts all the data access logic behind a dedicated class (the **repository**), allowing the rest of the application to work with data without knowing how or where it is stored. It did not hide only the database, it hides the entire data source.

Now if we change **EF-Core** with **Dapper** or **SQL** with **PostgreSql** or **MongoDb**, only the **repository** will be affected and changed. Everything else remains the same.

8. Unit of Work:

It is a designed pattern that groups multiple data operations (like insert, update, delete) made through one or more repositories and commits them as a single transaction. It ensures that either all operations succeed or all fail.

In **EF-Core** **AppDbContext** already does something similar. It tracks Added entities, Modified entities, Deleted entities. Then **SaveChanges()** commits them all together. People often say, **DbContext** is already a **UoF**. Because Microsoft designed it that way.

9. Entity Framework Core:

EF-Core is an open-source, light-weight, object-relational mapper(ORM) created by Microsoft for .NET applications. It acts as the translator or bridge between your C# object-oriented code and relational database.

Why Developers Use EF Core:

- **No More Raw SQL strings:** You query data using **C# LINQ** (Language Integrated Query) syntax instead of writing text-based **SQL** queries.
- **Database Agnostic:** EF Core works with **SQL Server**, **PostgreSQL**, **MySQL**, **SQLite**, **Cosmos DB**, and **Oracle**. You can switch the underlying database with just a line of configuration.
- **Automatic Table Creation (Migrations):** You write your **C#** models first, and **EF Core** automatically generates the database schema and columns for you.

How EF Core Connects to Your Patterns:

In the previous turns, we discussed **Repositories** and the **Unit of Work**. **EF Core** is built directly around those concepts right out of the box:

1. DbContext = Unit of Work:

The **DbContext** class in **EF Core** is your **Unit of Work**. It tracks all updates, inserts, and deletes you make to your objects in memory. When you call **_context.SaveChanges()**, it wraps all those pending changes into a single database transaction.

2. DbSet = Repository:

Properties in your context like **DbSet<User>** act as built-in repositories. They give you access to methods like **Add()**, **Remove()**, and **Find()** without you needing to create custom repository classes from scratch.

10. DbContext:

It is a primary class in **EF Core** which is responsible for managing database connections, track changes and perform **CRUD** operations. Think of it as a session with the database.

It is responsible for:

- Connecting to the database.
- Querying data.
- Tracking changes to entities.
- Saving changes back to the database.
- Managing transactions when needed.

11. Migrations:

Migration is EF-Core's way of tracking changes and applying them to the database schema over time. We do not manually write **SQL** like, **CreateTable()** or **AlterTable()** instead we only do changes in **C#** code and migrations tracks them and applies them to the database.

After creating a new class or modifying an existing class, we run the migrations command. Migrations tell **EF-Core** to compare the current model with the existing migrations if any and generate the **SQL** needed to make the database match.

Every migration file contains two methods:

- **Up**: Contains the code to apply the new changes to the database.
- **Down**: Contains the code to roll back those exact changes if something goes wrong.

12. Navigation Properties:

They are the properties in our entity classes that allow us to **navigate** between related entities using simple **C#** objects instead of manually working with **foreign keys**. Think of them as **object references** that represent **database relationships**.

There are two types of navigation properties:

1. **Reference Navigation:** Holds a reference to a single related entity (used by the dependent/child side).
2. **Collection Navigation:** Holds a list of related entities (used by the principal/parent side).

API & Backend

13. REST API:

A **REST API** (Representational State Transfer API) is a set of rules that allows different applications to communicate over **HTTP**. We can say it is a **contract** between **frontend** and **backend**. The **frontend** sends **HTTP** requests, the **backend** processes them, **interacts** with the database, and returns responses in **JSON** body with appropriate **HTTP** status codes.

REST API Best Practices:

- Use nouns for **URLs** (/students, /departments) rather than verbs.
- Use the correct **HTTP** method (**GET**, **POST**, **PUT**, **DELETE**) for the operation.
- Return meaningful HTTP status codes(**200 OK**, **201 Created**, **204 No Content**, **400 Bad Request**).
- Use **DTOs** instead of exposing **EF Core** entities directly.
- Keep **endpoints** focused on a **single** resource or action.

HTTP Resource:

In REST, everything is treated as a **resource**.

For Example:

Students, Departments, Courses etc.

14. DTOs (Data Transfer Objects):

A **DTO** (Data Transfer Object) is a simple object used to **transfer data between different layers or applications**.

In ASP.NET Core, DTOs are commonly used to transfer data:

- From the **Client (React)** to the **API**
- From the **API** to the **Client**
- Between the **Controller** and the **Application Layer**

A **DTO** contains only the data that **needs to be transferred**. It does not contain **business logic** or **database behavior**.

Benefits of DTO's:

1. Hide Sensitive Data:

Returning the entity directly would expose the sensitive information at frontend.

2. Reduce Response Size:

Instead of sending 20 properties, we send only required fields to the client.

3. Decouple API from Database:

Suppose we rename the attribute inside the entity. If clients use entities directly, every frontend breaks. While with DTO's, we simply map the entity field with the DTO's field. The API contract remains unchanged.

4. Prevent Over-Posting:

Suppose the entity has a field with a return type of bool. A client explicitly sends a “true” value inside a JSON body as a payload. If you bind directly to the entity, unintended properties could be modified. Instead if we use DTO’s, only allowed fields are accepted.

15. Validation:

Validation is the process of checking whether incoming data is **correct, complete, and follows the rules** before it is processed (apply business logic) or saved to the database.

Summary:

Concept	Description
Validation	Ensures incoming data meets required rules before processing.
Data Annotations	Attributes like [Required], [Range], and [EmailAddress] used to validate DTO properties.
ModelState	Contains validation results. Automatically checked when using [ApiController].
Business Validation	Rules that depend on application logic (e.g., unique email, existing department).
Client-Side Validation	Performed in the frontend for a better user experience.
Server-Side Validation	Performed in the API to ensure security and data integrity.

16. Pagination:

Pagination is the process of dividing a large set of data into **smaller pages**, so the client loads only a **limited number of records** at a time. Instead of **returning all records**, the **API** returns only the records for the **requested page**. Pagination is essential for building scalable APIs because it improves performance, reduces network traffic, and provides a smooth user experience when working with large datasets.

Two Types of Pagination:

1. Offset-Based Pagination (The Classic Way):

The **offset** is the number of records to skip before starting to return data.

- **How it works:** Uses Skip and Take.
- **Pros:** Easy to implement; users can jump directly to any page number they want (e.g., jump straight to Page 47).
- **Cons:** Becomes slow on massive tables because the database engine still has to scan through all skipped records internally before returning data.

2. Keyset / Cursor-Based Pagination (The Modern Way):

- **How it works:** Instead of counting skips, you filter by the last item's ID from the previous page (e.g., WHERE Id > lastSeenId LIMIT 10).
- **Pros:** Incredibly fast and scales infinitely regardless of database size. Perfect for infinite scrolling feeds (like Instagram or Twitter).
- **Cons:** Users cannot jump to specific arbitrary pages (like "Go to page 5").

Summary:

Concept	Description
Pagination	Divides large datasets into smaller pages.
page	The requested page number.
pageSize	Contains validation results. Automatically checked when using [ApiController].
Skip()	Skips records from previous pages.
Take()	Retrieves only the required number of records.
CountAsync()	Gets the total number of matching records.
OrderBy()	Ensures consistent ordering before pagination.

17. Sorting & Searching:

Sorting and **Searching** are two of the most common features in APIs. They allow users to:

- **Search:** Find specific records.
- **Sort:** Arrange records in a meaningful order.

They are almost always used together with **pagination**.

Summary:

Concept	Description
Searching	Filters records based on user input.
Sorting	Orders records by one or more fields.
Where()	Applies search filters.
OrderBy()	Sorts in ascending order.
OrderByDescending()	Sorts in descending order.
AsQueryable()	Allows building queries dynamically before execution.
Best Practice	Search → Sort → Count → Skip → Take

Architecture Patterns

18. CQRS (Command Query Responsibility Segregation):

CQRS (Command Query Responsibility Segregation) is an architectural pattern that separates **operations that modify data (Commands)** from **operations that read data (Queries)**.

The core principle is:

A request should either change data or return data, but not both.

Need of CQRS:

When everything is inside one service.

As the application grows:

- The service becomes large.
- Different responsibilities mix together.
- Testing becomes harder.
- Maintenance becomes more difficult.

CQRS separates these responsibilities.

Two Parts of CQRS:

1. Commands:

Commands **change data**.

Examples:

- Create Student
- Update Student
- Delete Student
- Create Department

Commands usually **do not return data** (other than a success indicator).

Examples:

- CreateStudentCommand
- UpdateStudentCommand
- DeleteStudentCommand

2. Queries:

Queries only **read data**. They **never modify data**.

Examples:

GetStudentsQuery

GetStudentByIdQuery

SearchStudentsQuery

CQRS with MediatR:

CQRS is often used with a mediator library such as **MediatR**.

Instead of:

Controller



StudentService

You have:

Controller



Mediator



Correct Handler

The mediator automatically routes each command or query to its corresponding handler.

Benefits:

1. Separation of Responsibilities
2. Smaller Classes
3. Easier Testing
4. Easier Maintenance
5. Easier Scaling

19. Command & Query Objects:

Command Objects and Query Objects are the core building blocks of **CQRS**.

They are simple classes (or records) that represent a single request.

- A **Command** Object represents an action that changes data.
- A **Query** Object represents a request to retrieve data.

They do not contain business logic. They only carry the data needed for the operation.

Command Object:

A command represents:

"Please perform this action."

Example:

```
public record CreateStudentCommand(  
    string Name,  
    string Email,  
    int Age,  
    int DepartmentId);
```

This object simply holds the data needed to create a student.

It does **not** create the student itself.

Query Object:

A query represents:

"Please return this data."

Example:

```
public record GetStudentByIdQuery(int Id);
```

It only contains:

- Student ID

It does **not** fetch the student.

20. Handler Pattern:

The **Handler Pattern** is a design pattern where each request is handled by its own dedicated class.

Instead of putting all business logic into **one large service**, each **operation** gets its own **Handler**.

In CQRS:

- Every **Command** has a **Command Handler**.
- Every **Query** has a **Query Handler**.

The handler contains the business logic needed to process that specific request.

Think of it as:

"The person responsible for processing one specific request."

Need of Handlers:

As more features are added in a traditional service:

- Hundreds of lines of code.
- Multiple responsibilities.
- Harder to test.
- Harder to maintain.

With the use of Handler pattern, these flaws get reduced because of one job per handler.

21. Repository Split (Read vs Write):

Repository Split is a **CQRS**-related approach where database operations are separated into two repositories:

Read Repository: responsible only for retrieving data.

Write Repository: responsible only for creating, updating, and deleting data.

The main idea is the same as CQRS:

Separate reading data from modifying data.

Event-Driven Architecture

22. Event-Driven Architecture:

Event-Driven Architecture (EDA) is a design pattern where components communicate by producing and consuming events instead of calling each other directly.

In a traditional approach, when a student is created, the service directly calls every other component that needs to know, PostgreSQL sync, email sender, logger, etc. This creates tight coupling. If any one of them fails, the whole operation fails.

In **EDA**, the service simply says "**a student was created**" and publishes an event. It doesn't know or care who is listening. Any interested component picks it up independently and reacts in its own time.

The real goal of **EDA** is: "**producers and consumers should not know about each other.**"

23. Integration Events:

Integration events are plain **C#** objects that carry the data describing something that happened in the system. They are the actual "message" that gets published to the queue.

In our project:

- **StudentCreatedEvent** carries Id, Name, Email, Age, DepartmentId
- **StudentDeletedEvent** carries only Id

They implement a shared marker interface **IIntegrationEvent** so the queue can accept any event type without knowing specifically what it is.

The rule: The **event** should contain enough information for the **consumer** to do its job without going back to the **producer's** database.

24. Producer-Consumer Pattern:

Producer-Consumer is a concurrency pattern where one part of the system produces work items and puts them in a shared queue, while another part independently consumes and processes them.

In our project:

- **Producer:** **AddStudentCommandHandler**, **UpdateStudentCommandHandler**, **DeleteStudentCommandHandler**, they publish events to the queue after a successful SQL Server write.
- **Queue:** **InMemoryEventQueue**, the shared buffer between producer and consumer.
- **Consumer:** **PostgresSyncWorker**, sits in a loop, pulls events off the queue one by one, and applies them to **PostgreSQL**.

The key benefit: The producer never waits for the consumer. It drops the event and immediately returns success to the client.

25. In-Memory Queue:

An in-memory queue is a data structure that holds items temporarily inside the application's memory, following a **First-In-First-Out (FIFO)** order, the first event published is the first one processed.

In our project, we used **System.Threading.Channels** (a built-in .NET feature that provides a way to build **producer-consumer pipelines**.) to implement it.

It is designed exactly for producer-consumer scenarios: **Enqueue** writes to the channel, **DequeueAsync** reads from it and genuinely waits (without **polling** or **sleeping**) until an item arrives.

The trade-off: Since it lives in memory, if the **application crashes**, any unprocessed events in the **queue are lost**. For production systems, external brokers like **RabbitMQ** or **Kafka** are used instead.

26.IHostedService:

IHostedService is a built-in .NET interface used to run **background work** while the application is running.

It has two methods:

1. **StartAsync(CancellationToken)**
 2. **StopAsync(CancellationToken)**
- **StartAsync** runs when the application starts.
 - **StopAsync** runs when the application is shutting down.

How it works

The important thing to remember is that **StartAsync** should **start the background work and return quickly**.

We should **not** put a long-running loop directly inside **StartAsync** and **await** it.

For example, if the worker has a loop that keeps running:

```
while the application is running
  do some work
  wait
  do some work
  wait
  ...
```

and we **await** that loop inside **StartAsync**, the application will keep waiting for the loop to finish.

But the loop is supposed to run until the application shuts down, so it would never finish while the application is running.

Instead, we start the loop as a separate background **Task**:

```
Application starts
  ↓
StartAsync()
  ↓
Start background task
  ↓
Return immediately
  ↓
Application continues running
```

We keep a reference to that task so we can stop it later.

Stopping the worker

When the application shuts down, **StopAsync** is called.

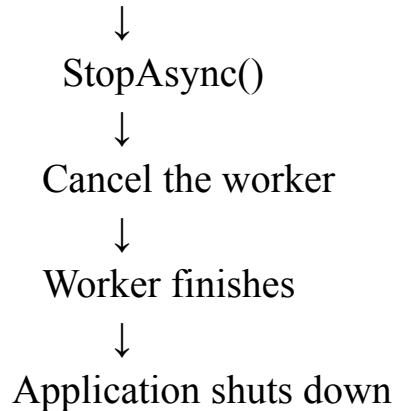
It tells the background task:

"The application is shutting down. Please stop."

This is done using a **CancellationTokenSource**.

Then we **await** the background task so the application gives it time to finish its current work properly.

Application shutting down



In our project

PostgresSyncWorker directly implements **IHostedService**.

That means it is responsible for:

- Starting its background work in **StartAsync**
- Stopping its background work in **StopAsync**
- Managing its own **CancellationTokenSource**
- Keeping track of the background **Task**

So, in simple terms:

IHostedService lets our application start a background worker when it starts and stop that worker properly when it shuts down.

27. BackgroundService:

BackgroundService is a **built-in .NET class** that makes it easier to create background workers.

It already implements **IHostedService** for us.

With **IHostedService**, we have to write:

StartAsync()

StopAsync()

and manually manage things like:

- Starting the background task
- Storing the task
- Cancelling the task when the application shuts down
- Waiting for the task to finish

BackgroundService does this work for us.

Instead, we mainly need to implement one method:

**protected abstract Task ExecuteAsync(
CancellationToken stoppingToken);**

This method contains the actual background work.

For example:

```
protected override async Task ExecuteAsync(
    CancellationToken stoppingToken)
{
    while (!stoppingToken.IsCancellationRequested)
    {
        await DoWorkAsync();
    }
}
```

What happens behind the scenes?

When the application starts:

Application starts



BackgroundService starts



ExecuteAsync() runs



Background work continues

When the application shuts down:

Application shutting down



BackgroundService cancels the token



ExecuteAsync() sees the cancellation



Background work stops

So **BackgroundService** is **not magic**.

It simply contains the **IHostedService** logic that we would otherwise have to write ourselves.

Think of it like this:

IHostedService



We write the plumbing ourselves



StartAsync + StopAsync + Task + Cancellation

Whereas:

BackgroundService



.NET provides the plumbing



We mainly write ExecuteAsync()

In our project

We first used **BackgroundService** because it is the easier and more common way to create a background worker.

Later, we changed **PostgresSyncWorker** to implement **IHostedService** directly.

The purpose was to understand **what BackgroundService is actually doing behind the scenes**.

So the easiest way to remember it is:

IHostedService gives us the basic building blocks for a background worker. **BackgroundService** is a convenient class that handles those building blocks for us, so we mainly need to write **ExecuteAsync()**.

28. CancellationToken:

A **CancellationToken** is a way to tell a running piece of code that it should stop.

It is like a "please stop" signal.

It does not suddenly kill the code or force a thread to stop. Instead, the code gets the signal and **stops itself safely**.

Think of it like telling a worker:

"We're closing the office. Finish what you're doing, then stop."

How does it work?

The running code receives a **CancellationToken**:

CancellationToken stoppingToken

It can check:

stoppingToken.IsCancellationRequested

If this is:

false

the application is still running, so the worker continues.

If it becomes:

true

the application is shutting down, so the worker should stop.

For example:

```
while (!stoppingToken.IsCancellationRequested)
{
    await DoWorkAsync();
}
```

This means:

"Keep doing the work while nobody has asked me to stop."

What happens when the application shuts down?

The application tells the worker:

"Please stop."

↓

CancellationToken is triggered

↓

IsCancellationRequested = true

The worker notices this and exits its loop.

Application shutting down



CancellationToken triggered



Worker notices the signal



Worker stops safely

DequeueAsync(stoppingToken)

In our project, the worker waits for events from the queue:

```
await DequeueAsync(stoppingToken);
```

The token is passed to **DequeueAsync** so that the waiting operation can also be cancelled.

If the application shuts down while the worker is waiting for an event, the cancellation token is triggered.

DequeueAsync then throws:

`OperationCanceledException`

This tells the worker:

"The application is shutting down. Stop waiting for more events."

The worker can catch this exception and exit cleanly.

Why is this important?

Imagine the worker is processing data for PostgreSQL.

We don't want the application to suddenly kill the worker in the middle of a database operation.

Instead:

Application shutting down



Cancellation requested



Worker finishes/handles current work



Worker stops



Application shuts down

This is called **cooperative cancellation** because the code cooperates with the cancellation request rather than being forcefully terminated.

In our project

PostgresSyncWorker receives:

CancellationToken stoppingToken

The worker uses it in two important places:

1. Checking whether it should continue:

```
while (!stoppingToken.IsCancellationRequested)
```



```
{  
    // work  
}
```

2. Cancelling the queue wait:

```
await DequeueAsync(stoppingToken);
```

So the basic idea is:

CancellationToken is a polite stop signal. The worker checks the signal, finishes safely, and then exits instead of being forcefully killed.

Database

29. SQL Server:

It stores data in tables with rows and columns, and uses SQL (Structured Query Language) to query and manipulate that data.

In our project, SQL Server acts as the **write database** — the single source of truth. Every Create, Update, and Delete operation hits SQL Server first. It is responsible for generating Ids (via its identity column), enforcing constraints, and maintaining the authoritative, up-to-date state of the data.

We connected to it using EF Core with the **Microsoft.EntityFrameworkCore.SqlServer** package and a dedicated **SqlServerDbContext**.

30. PostgreSQL:

PostgreSQL (also called Postgres) is a powerful, open-source relational database management system known for its reliability, standards compliance, and performance. Like SQL Server, it stores data in tables and uses SQL.

In our project, PostgreSQL acts as the **read database** — a replica of SQL Server, kept in sync asynchronously via the event-driven architecture. Client-facing read operations (**GetAll**, **GetById**) are served from Postgres via **IStudentReadRepository** and **IDepartmentReadRepository**.

We connected to it using EF Core with the **Npgsql.EntityFrameworkCore.PostgreSQL** package and a dedicated **PostgreSqlDbContext**. Since Postgres never generates its own Ids (SQL Server owns Id generation), we configured **.ValueGeneratedNever()** on all primary keys in **PostgreSqlDbContext**.

31. Dual Database Architecture:

Dual Database Architecture is a setup where two separate databases serve different purposes within the same system, rather than one database handling everything.

In our project:

- **SQL Server** = Write DB (Commands go here — Create, Update, Delete)
- **PostgreSQL** = Read DB (Queries come from here — GetAll, GetById)

This maps directly to CQRS at the infrastructure level: Commands use **IStudentRepository** → **SqlServerDbContext**, Queries use **IStudentReadRepository** → **PostgreSqlDbContext**.

The benefit: reads and writes can scale independently, and the read database can be optimized specifically for query performance without affecting the write side.

The trade-off: the two databases are not instantly in sync — PostgreSQL is eventually consistent with SQL Server, with a small delay introduced by the asynchronous event-driven sync.

32. Eventually Consistent Systems:

Eventually consistent means that when data is changed, all copies of that data will **eventually become up to date**, but they may not update at exactly the same time.

In our project:

1. Student saved to SQL Server



2. Event added to queue



3. PostgresSyncWorker processes it



4. Student saved to PostgreSQL

There may be a small delay between step 1 and 4.

So for a short time:

SQL Server → Student exists

PostgreSQL → Student doesn't exist yet

This is acceptable because PostgreSQL is mainly used for **read/display operations**.

The API also doesn't wait for PostgreSQL. Once SQL Server successfully saves the student, the API can return success.

Strong Consistency:

Strong consistency would require both databases to be updated before returning success:

SQL Server

+

PostgreSQL

↓

Return success

This would be slower and could cause the whole operation to fail if PostgreSQL is temporarily unavailable. Which is exactly the problem event-driven architecture was introduced to solve.

Frontend

33. React:

React is an open-source JavaScript library developed by Meta (Facebook) for building user interfaces.

React is **component-based**, which means the UI is divided into small, reusable pieces called **components**.

For example:

App

└─ DepartmentList

└─ DepartmentModal

└─ StudentList

└─ StudentModal

React also uses a **Virtual DOM**. Instead of changing the browser's real DOM every time something changes, React first compares the new UI with the previous one and updates only the parts that actually changed. This makes UI updates more efficient.

In our project, React is used for the **frontend**. It handles:

- Students and Departments
- CRUD operations
- Search

- Pagination
- Sorting
- Form validation

It communicates with our **ASP.NET Core API** through HTTP requests.

So, simply:

React builds the frontend UI, manages user interactions, and communicates with the backend API to display and update data.

34. React Hook Form:

React Hook Form (RHF) is a lightweight library for managing form state and validation in React. It is built around React hooks and follows an **uncontrolled input** approach, instead of storing every keystroke in React state (which causes re-renders on every keypress), it registers inputs directly with the DOM and only reads their values when needed (on submit). This makes it significantly more performant than controlled form approaches.

In our project, RHF manages all Student and Department forms. Handling input registration, form submission, error display, and reset. It integrates directly with Zod for schema-based validation via the **zodResolver**.

35. Zod Validation:

Zod is a TypeScript-first schema declaration and validation library for JavaScript. You define a schema describing the shape and rules of your data, and Zod validates any input against that schema — returning either a success with parsed data or a list of validation errors.

In our project, Zod is used on the frontend to validate form inputs before they are submitted to the API. For example, ensuring **Name** is not empty, **Age** is a valid number, and **Email** matches a proper format. It handles tricky edge cases like **NaN** (from empty number inputs) and **undefined** (from unselected dropdowns) using **z.coerce** and **z.preprocess**.

Zod integrates with React Hook Form via **zodResolver**, which connects the schema's validation rules directly to RHF's error handling — so validation errors automatically appear on the right fields without extra wiring.

36. Mantine UI:

Mantine is a React component library that provides ready-made UI components for building modern interfaces.

Instead of creating everything from scratch with HTML and CSS, we can use components such as:

- Button
- Modal
- Table
- TextInput
- Select
- Pagination

For example:

```
<Button>Add Student</Button>
```


In our project, we use Mantine for the **frontend UI**, including:

- Student and Department tables
- Add/Edit modals
- Delete confirmation modal
- Buttons and inputs
- Pagination
- Form layout and styling

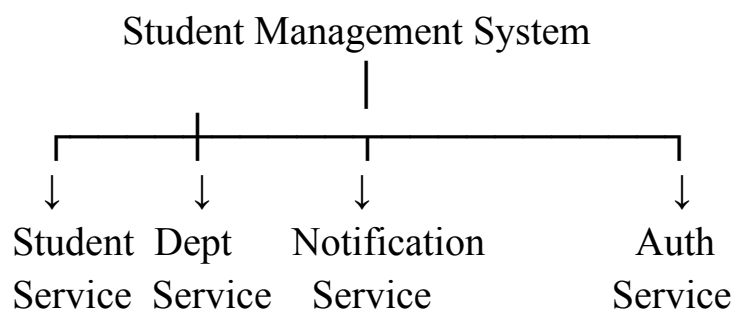
So, simply:

Mantine provides ready-made React UI components, making it easier and faster to build the interface without writing all the styling from scratch.

37. Microservices:

Microservices is an architectural style where a large application is divided into **small, independent applications**, with each one responsible for a specific business function.

For example, instead of one Student Management System:



Each service:

- Has its own code
- Usually has its own database
- Runs independently
- Communicates with other services through APIs or message brokers
- Can be deployed and scaled separately

For example:

Student Service



Handles students only

Department Service



Handles departments only

Notification Service



Handles emails and alerts

Auth Service



Handles authentication

Is Microservices Always Better?

No.

Microservices are not automatically better architecture.

For a small application or small team, they can add unnecessary complexity:

- Network communication
- Authentication between services
- Distributed debugging and tracing
- Deployment complexity
- Network latency
- Multiple databases to manage

The Main Idea:

Microservices means splitting an application into independently deployable services, where each service focuses on one specific business responsibility.

The goal isn't to have more services. The goal is to use **the right architecture for the size and needs of the system.**

38. Dependency Inversion vs Dependency Injection:

These two are related, but they are **not the same thing**.

Think of it this way:

DIP is the rule. DI is the way we follow that rule.

1. Dependency Inversion Principle (DIP)

DIP is a SOLID principle.

It says:

Depend on abstractions, not concrete classes.

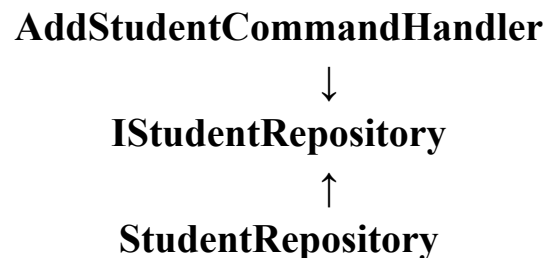
In our project, the handler doesn't directly depend on:

StudentRepository

Instead, it depends on:

IStudentRepository

So:



The handler knows **what it needs** through the interface, without caring about the specific implementation.

That's **DIP**.

2. Dependency Injection (DI)

DI is the technique used to provide those dependencies.

Instead of the handler doing this:

```
var repository = new StudentRepository();
```

the dependency is provided from outside:

```
public AddStudentCommandHandler(  
    IRepository repository)  
{  
    _repository = repository;  
}
```

.NET's **DI container** creates the required objects and gives them to the handler.

That's **DI**.

Easy way to remember:

DIP = What should I depend on? → An abstraction.

DI = How do I get that dependency? → From outside the class.